



HASHDIT

HashDit

Smart Contract Security

ListaDao MarketFactory bStock Update

Security Audit Report

ListaDao

24 Aug 2026 – 26 Aug 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	24 Aug 2026 - 26 Aug 2026
Overall Risk	Low
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/231
Commit	feat/market-factory-stock-and-liquidation-vault

Smart Contract List

No.	Contract Name	Verdict	Details
1	MarketFactory.sol	Low	[L01] [I01] [I02] [I03] [I04]

Findings

[L01] Factory liquidationVault pointer can drift from the liquidators fundSource

Contract MarketFactory.sol

Severity Low

Description

The MarketFactory tracks a liquidationVault address that it uses to whitelist each new market's tokens on the shared vault. Separately, each Liquidator and BrokerLiquidator tracks its own fundSource address, which is the vault that liquidation proceeds are reflowed into and that is granted pull rights over the liquidator. These two pointers are independent and nothing on chain keeps them equal. The factory only writes token whitelists against its own liquidationVault and never configures fundSource. setFundSource on the liquidators is a MANAGER action that accepts address(0) and can be repointed to a different vault, while the factory's setLiquidationVault rejects address(0).

JavaScript

```
function setLiquidationVault(address _liquidationVault) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    require(_liquidationVault != address(0), "ZeroAddress");
    emit LiquidationVaultUpdated(address(liquidationVault),
    _liquidationVault);
    liquidationVault = ILiquidationVault(_liquidationVault);
}
```

Description

Impact

If the two values diverge, for example the liquidators' fundSource is repointed to a new vault or disabled without updating the factory's liquidationVault, or the reverse, the factory keeps whitelisting newly created markets' tokens on a stale or wrong vault. The vault that the liquidators actually reflow into is then missing those token entries, so its sellToken, sellBNB, and collectERC20 calls revert with NotWhitelisted for those tokens until a MANAGER whitelists them by hand.

Reflow still transfers balances unconditionally to whatever fundSource points to, so proceeds can accumulate in that vault and remain unsellable until the whitelist is corrected.

Because setLiquidationVault cannot be set to address(0), the factory side also cannot mirror a disabled fundSource on the liquidator contracts side and will keep configuring a vault the liquidators no longer use. No funds are lost, since a MANAGER can still rescue balances, but liquidation processing can stall and configuration can silently target the wrong vault.

Recommendation	Either derive the vault the factory whitelists against from the liquidators' fundSource so there is a single source of truth, or add validation and operational documentation that require both to be updated in the same procedure. Also consider allowing setLiquidationVault to accept address(0) so the factory can mirror a disabled fundSource and stop configuring an unused vault.
Status	[Open/Fixed/Acknowledged]

[I01] Unvalidated stockCollateral flag can silently disable market hours gating for bStock markets

Contract MarketFactory.sol

Severity **Informational**

Description

The MarketFactory decides whether a newly created market receives market hours protection based entirely on the stockCollateral boolean that the OPERATOR supplies at creation time. When a bStock market is created (its param.oracle is a StockOracle) but stockCollateral is passed as false, the collateral token is never registered in the StockOracleSwitch.

Description

JavaScript

```
// if collateral is a tokenized stock, register it in the
// market-hours switch. setStock also opens
// the stock; it stays gated by the switch's global market-hours
// flag, which MANAGER owns.
if (stockCollateral) {
  require(address(stockOracleSwitch) != address(0),
    "StockOracleSwitch not set");
  if (!stockOracleSwitch.registered(param.collateralToken)) {
    stockOracleSwitch.setStock(param.collateralToken, true);
  }
}
```

Because StockOracleSwitch.isEnabled returns true for any token that is not registered, an unregistered token is treated as passthrough and is always considered enabled. As a result StockOracle.peek never gates the price, and the market keeps trading continuously using the last Atlas price, which can be stale while the underlying stock market is closed. No revert or error is produced, so the missing protection is completely silent. The factory never cross checks the flag against the oracle, so nothing on chain detects the mismatch.

JavaScript

```
function isEnabled(address token) external view returns (bool) {
```

```

    if (!registered[token]) return true; // not a managed stock ->
    passthrough (always enabled)
    return globalEnabled && enabled[token]; // registered -> enabled
    only if the market and the stock are on
  }

```

Impact

The impact is that borrowing and liquidation can proceed against stale, closed market prices, which can create bad debt or trigger unfair liquidations, defeating the exact protection this feature is meant to provide. The severity is capped at Low because triggering it requires a mistake by the trusted OPERATOR, but the failure is silent and leaves no on chain trace.

Recommendation

Derive whether the collateral is a tokenized stock instead of trusting a bare flag. For example, have the factory detect that param.oracle routes through the wired StockOracleSwitch and register the collateral accordingly.

If the flag must remain, ensure that the OPERATOR does not make such a mistake by always enforcing that stockCollateral = true during a creation of a bStock market.

Status

[Open/Fixed/Acknowledged]

[I02] Wiring the liquidation vault or oracle switch without granting the factory the MANAGER role blocks all market creation

Contract MarketFactory.sol

Severity **Informational**

Description

Description

The MarketFactory exposes setter functions that configure the liquidation vault address and the stock oracle switch address. These setters only store the supplied address and do not confirm that the factory has been granted the MANAGER role on the target contracts.

JavaScript

```

    if (includeVault && address(liquidationVault) != address(0) &&
    !liquidationVault.tokenWhitelist(token)) {
      liquidationVault.setTokenWhitelist(token, true);
    }

```

Impact

Once an address is wired, every call to createMarket invokes liquidationVault.setTokenWhitelist() for the loan token of each common and fixed term market (through the internal token whitelisting helper), and stock markets additionally call stockOracleSwitch.setStock(). Both of these external calls require the factory to hold the MANAGER role on the respective contract. If an administrator sets the address but the role has not been granted, these calls revert, which prevents creation of all affected markets. The existing address(0) guard only handles the case where the dependency is not yet deployed, and it does not cover the case where the address is wired but the role is still missing.

Recommendation

Consider ensuring the granting of the MANAGER role to the factory in the same administrative transaction or deployment runbook that wires the liquidation vault and oracle switch addresses.

Additionally, consider a dedicated script to perform such actions.

Status

[Open/Fixed/Acknowledged]

[I03] Common market tokens are whitelisted on the BrokerLiquidator but can never be used there

Contract

MarketFactory.sol

Severity

Informational

Description

The refactored token whitelisting helper routes every common market's loan and collateral tokens onto the BrokerLiquidator tokenWhitelist during market creation. Common markets can never be liquidated through the BrokerLiquidator.

Every liquidation entry in that contract, including flashLiquidate, liquidate, liquidateSmartCollateral, and flashLiquidateSmartCollateral, resolves the market's broker and requires marketIdToBroker[id] != address(0). A market is only registered in marketIdToBroker through setMarketToBroker, which the factory calls only when creating fixed term broker markets, never for common markets.

Description

JavaScript

```
function _whitelistToken(address token, bool includeVault) private {
    if (!liquidator.tokenWhitelist(token)) {
        liquidator.setTokenWhitelist(token, true);
    }
    if (address(brokerLiquidator) != address(0) &&
        !brokerLiquidator.tokenWhitelist(token)) {
        brokerLiquidator.setTokenWhitelist(token, true);
    }
    if (includeVault && address(liquidationVault) != address(0) &&
        !liquidationVault.tokenWhitelist(token)) {
        liquidationVault.setTokenWhitelist(token, true);
    }
}
```

```
}  
}
```

Impact

As a result a common market's id maps to address(0) and all BrokerLiquidator liquidation paths revert for it. The token whitelist entries created for common markets on the BrokerLiquidator are therefore inert. They are harmless, since the sell and collect legs are BOT gated and no common market liquidation can reach them, but they widen the allow list surface with entries that are never exercised.

TBD

Recommendation

Consider adding a brokerMarket flag to _whitelistToken and route to the associated liquidator only (same for fixed term market creation)

Status

[Open/Fixed/Acknowledged]

[I04] LiquidationVault LP exclusion is achieved by omission and can be re-enabled by any market

Contract

MarketFactory.sol

Severity

Informational

Description

The LiquidationVault must never whitelist the smart-collateral LP token. The code comment states this directly, noting that whitelisting the LP in the vault would open the BOT collect* path for it. However, this exclusion is enforced only by omission by trusting the OPERATOR role in passing includeVault=false simply skips the vault write during token whitelisting. There is no LiquidationVault side deny mechanism, given the existing reflowBlacklist lives only on the liquidators, where it guards _reflow and not collectERC20.

Description

JavaScript

```
// token whitelist across the liquidation suite (Liquidator /  
BrokerLiquidator / LiquidationVault).  
// A smart-collateral LP is excluded from the vault only: the vault  
cannot sell it, so it is  
// reflow-blacklisted in _configSmartProvider and only its  
underlying pair legs are vault-whitelisted.  
// Whitelisting the LP there would also open the BOT collect* path  
for it.  
_whitelistToken(param.loanToken, true);  
_whitelistToken(param.collateralToken, !liquidatorSmartProvider);
```

Because the token whitelist flag is global and can be set by any market, the intended exclusion breaks the moment the same LP token is passed to `_whitelistToken(..., true)` from a different market creation path. This can happen when

- A second market uses that LP as plain collateral with `liquidatorSmartProvider=false`,
- When that token appears as a loan token anywhere
- When a fixed-term market with that collateral is created, since fixed-term creation always uses `includeVault=true`.

Impact

The outcome is order independent. If the smart market is created first, the vault flag stays false, and then the other market's guard sees false and sets it to true. If the other market is created first, the vault flag is already true, and the smart market's `includeVault=false` leaves it true. In both cases the end state is `vault.tokenWhitelist[LP] = true`.

Once the LP is vault whitelisted, the BOT `collectERC20(liquidator, LP, amount)` call passes its gate and pulls the LP out of the Liquidator into the vault, which is exactly the scenario the reflow blacklist was built to prevent. Normally the LP sits in the Liquidator after a wrong entry plain liquidate and is recoverable through `redeemSmartCollateral`. After it is collected into the vault it becomes stuck, because the vault cannot sell it and the redeem path no longer applies, leaving recovery possible only through a manual `MANAGER withdrawERC20`.

Recommendation

Consider introducing a vault side deny set, a permanent per token exclusion checked inside `collectERC20` and `sellToken`, and populate it for the smart-collateral LP when the smart market is created. Have `_whitelistToken` respect this deny set so that no later market creation path can flip `vault.tokenWhitelist[LP]` back to true, either by skipping the vault write for denied tokens or by reverting when a denied token is presented for vault whitelisting regardless of the `includeVault` flag.

If not, for the above 3 scenarios, ensure that the OPERATOR performing the creation is enforced to always use appropriate inputs when performing a smart collateral market creation.

Status

[Open/Fixed/Acknowledged]